

BIBLE OPENCV (CV2) – RÉFÉRENCE EXHAUSTIVE DU MODULE

LÉGENDE : [img] = image (ndarray) | [vid] = VideoCapture | [fn] = fonction
[draw] = dessin | [feat] = features | [ml] = machine learning

[img] Lecture, écriture et affichage – imread / imwrite / imshow

DESCRIPTION : Charger, sauvegarder et afficher des images.
POURQUOI : Point d'entrée de tout pipeline de vision par ordinateur.
CONTEXTE : JPEG, PNG, BMP, TIFF, WebP, et tout format supporté.

LECTURE

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")          Charge en BGR (pas RGB !)
img = cv2.imread("image.jpg",          BGR 3 canaux (défaut)
    cv2.IMREAD_COLOR)
img = cv2.imread("image.jpg",          Niveaux de gris (1 canal)
    cv2.IMREAD_GRAYSCALE)
img = cv2.imread("image.jpg",          Avec canal alpha si présent
    cv2.IMREAD_UNCHANGED)
img = cv2.imread("image.png",          16 ou 32 bits
    cv2.IMREAD_ANYDEPTH)
img = cv2.imdecode(np.frombuffer(bytes_data,
    np.uint8), cv2.IMREAD_COLOR)      Depuis des bytes en mémoire
```

ÉCRITURE

```
cv2.imwrite("sortie.jpg", img)          Sauvegarde en JPEG
cv2.imwrite("sortie.png", img)          Sauvegarde en PNG
cv2.imwrite("sortie.jpg", img,          Qualité JPEG (0-100)
    [cv2.IMWRITE_JPEG_QUALITY, 95])
cv2.imwrite("sortie.png", img,          Compression PNG (0-9)
    [cv2.IMWRITE_PNG_COMPRESSION, 9])
cv2.imwrite("sortie.webp", img,          Qualité WebP
    [cv2.IMWRITE_WEBP_QUALITY, 80])
_, buffer = cv2.imencode(".jpg", img)   Encode en bytes en mémoire
_, buffer = cv2.imencode(".jpg", img,
    [cv2.IMWRITE_JPEG_QUALITY, 85])
bytes_data = buffer.tobytes()           Convertit en bytes Python
```

AFFICHAGE

```
cv2.imshow("fenêtre", img)             Affiche dans une fenêtre
cv2.waitKey(0)                         Attend une touche (0=infini)
cv2.waitKey(1)                         Attend 1ms (pour les vidéos)
cv2.waitKey(100)                       Attend 100ms
key = cv2.waitKey(0) & 0xFF             Récupère le code de la touche
cv2.destroyAllWindows()                Ferme toutes les fenêtres
cv2.destroyWindow("fenêtre")           Ferme une fenêtre spécifique
cv2.namedWindow("fenêtre",            Fenêtre redimensionnable
    cv2.WINDOW_NORMAL)
cv2.namedWindow("fenêtre",            Taille fixe (défaut)
    cv2.WINDOW_AUTOSIZE)
cv2.resizeWindow("fenêtre", 800, 600)  Redimensionne la fenêtre
cv2.moveWindow("fenêtre", 100, 100)    Déplace la fenêtre
```

ATTRIBUTS D'UNE IMAGE

```
img.shape                             (hauteur, largeur, canaux) ou (h, w)
img.shape[:2]                         (hauteur, largeur)
img.height = img.shape[0]              Hauteur en pixels
img.width = img.shape[1]               Largeur en pixels
img.dtype                             Type de données (uint8, float32...)
img.size                              Nombre total de pixels × canaux
img.ndim                              Nombre de dimensions (2 ou 3)
len(img.shape) == 2                    True si image en niveaux de gris
```

ATTENTION BGR vs RGB

```
# OpenCV charge en BGR, pas RGB !
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  Convertir BGR → RGB (pour PIL/plt)
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)  Convertir RGB → BGR
```

EXEMPLES :

```
img = cv2.imread("photo.jpg")
```

```

if img is None:
    raise FileNotFoundError("Image introuvable")
h, w = img.shape[:2]
print(f"{w}x{h} px, {img.dtype}")

cv2.imshow("Photo", img)
if cv2.waitKey(0) & 0xFF == ord("q"):
    cv2.destroyAllWindows()

```

[img] Conversions de couleur – cvtColor

DESCRIPTION : Convertir entre espaces colorimétriques.
 POURQUOI : Segmentation HSV, traitement en gris, détection de couleurs.

CONVERSIONS COURANTES :

cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)	BGR → Niveaux de gris
cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)	Gris → BGR (3 canaux identiques)
cv2.cvtColor(img, cv2.COLOR_BGR2RGB)	BGR → RGB
cv2.cvtColor(img, cv2.COLOR_RGB2BGR)	RGB → BGR
cv2.cvtColor(img, cv2.COLOR_BGR2HSV)	BGR → HSV
cv2.cvtColor(img, cv2.COLOR_HSV2BGR)	HSV → BGR
cv2.cvtColor(img, cv2.COLOR_BGR2HLS)	BGR → HLS
cv2.cvtColor(img, cv2.COLOR_HLS2BGR)	HLS → BGR
cv2.cvtColor(img, cv2.COLOR_BGR2LAB)	BGR → CIE L*a*b*
cv2.cvtColor(img, cv2.COLOR_LAB2BGR)	L*a*b* → BGR
cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)	BGR → YCrCb
cv2.cvtColor(img, cv2.COLOR_YCrCb2BGR)	YCrCb → BGR
cv2.cvtColor(img, cv2.COLOR_BGR2XYZ)	BGR → CIE XYZ
cv2.cvtColor(img, cv2.COLOR_BGRA2BGR)	BGRA → BGR (supprime alpha)
cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)	BGR → BGRA (ajoute alpha)

VALEURS HSV DANS OPENCV :

```

# H : 0-179 (pas 0-360 !), S : 0-255, V : 0-255
rouge_bas = np.array([0, 120, 70])
rouge_haut = np.array([10, 255, 255])
# Rouge couvre aussi 170-179
vert_bas = np.array([40, 40, 40])
vert_haut = np.array([80, 255, 255])
bleu_bas = np.array([100, 150, 0])
bleu_haut = np.array([140, 255, 255])

```

[img] Redimensionnement et transformation géométrique – resize / warpAffine

DESCRIPTION : Modifier la taille, rotation, perspective d'une image.
 POURQUOI : Normalisation pour ML, correction de perspective, augmentation de données.

— REDIMENSIONNEMENT —

cv2.resize(img, (largeur, hauteur))	Taille exacte
cv2.resize(img, (640, 480))	640x480 pixels
cv2.resize(img, None, fx=0.5, fy=0.5)	50% de la taille
cv2.resize(img, None, fx=2.0, fy=2.0)	Doubler la taille
cv2.resize(img, (w, h), interpolation=cv2.INTER_LINEAR)	Interpolation bilinéaire (défaut agrandissement)
cv2.resize(img, (w, h), interpolation=cv2.INTER_AREA)	Interpolation par aire (défaut réduction)
cv2.resize(img, (w, h), interpolation=cv2.INTER_CUBIC)	Bicubique (meilleure qualité)
cv2.resize(img, (w, h), interpolation=cv2.INTER_NEAREST)	Plus proche voisin (rapide)
cv2.resize(img, (w, h), interpolation=cv2.INTER_LANCZOS4)	Lanczos (meilleure pour réduction)

— FLIP / ROTATION —

cv2.flip(img, 0)	Retournement vertical (axe X)
cv2.flip(img, 1)	Retournement horizontal (axe Y)
cv2.flip(img, -1)	Retournement des deux axes
cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)	Rotation 90° horaire
cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)	Rotation 90° antihoraire
cv2.rotate(img, cv2.ROTATE_180)	Rotation 180°

— ROTATION ARBITRAIRE —

```
h, w = img.shape[:2]
cx, cy = w // 2, h // 2
M = cv2.getRotationMatrix2D((cx, cy), angle=45, scale=1.0)
rot = cv2.warpAffine(img, M, (w, h))
```

— TRANSFORMATION AFFINE —

```
pts_src = np.float32([[0,0],[w,0],[0,h]])
pts_dst = np.float32([[0,0],[w,0],[30,h]])
M = cv2.getAffineTransform(pts_src, pts_dst)
warped = cv2.warpAffine(img, M, (w, h))
```

— TRANSFORMATION DE PERSPECTIVE —

```
pts_src = np.float32([[x1,y1],[x2,y2],[x3,y3],[x4,y4]]) # 4 coins source
pts_dst = np.float32([[0,0],[w,0],[0,h],[w,h]]) # 4 coins destination
M = cv2.getPerspectiveTransform(pts_src, pts_dst)
result = cv2.warpPerspective(img, M, (w, h))
```

— RECADRAGE —

```
crop = img[y1:y2, x1:x2] # Recadrage par slicing NumPy
roi = img[100:300, 200:500] # Région d'intérêt (ROI)
```

EXEMPLES :

```
# Redimensionner en conservant le ratio
h, w = img.shape[:2]
cible_w = 800
cible_h = int(h * cible_w / w)
redim = cv2.resize(img, (cible_w, cible_h), interpolation=cv2.INTER_AREA)

# Correction de perspective (document scanné)
# Détecter les 4 coins puis appliquer warpPerspective
```

[img] Filtrage et flou – blur / GaussianBlur / Canny
--

DESCRIPTION : Appliquer des filtres de lissage, netteté, détection de contours.
POURQUOI : Débruitage, prétraitement, détection de features.

— FLOU —

cv2.blur(img, (5, 5))	Flou moyen (kernel 5x5)
cv2.GaussianBlur(img, (5, 5), 0)	Flou gaussien (kernel impair)
cv2.GaussianBlur(img, (15, 15), sigmaX=2)	Avec sigma explicite
cv2.medianBlur(img, 5)	Flou médian (bon anti-bruit sel/poivre)
cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)	Flou bilatéral (préserve les bords)
cv2.stackBlur(img, (15, 15))	Stack blur (rapide, 4.7+)
cv2.boxFilter(img, -1, (5, 5))	Filtre boîte normalisé

— NOYAUX PERSONNALISÉS —

```
kernel = np.ones((5,5), np.float32) / 25 # Convolution avec kernel custom
cv2.filter2D(img, -1, kernel)

# Netteté (sharpening)
kernel_sharp = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
cv2.filter2D(img, -1, kernel_sharp)

# Emboss
kernel_emboss = np.array([[ -2,-1,0],[-1,1,1],[0,1,2]])
cv2.filter2D(img, -1, kernel_emboss)
```

— MORPHOLOGIE —

kernel = np.ones((5,5), np.uint8)	
cv2.erode(img, kernel, iterations=1)	Érosion
cv2.dilate(img, kernel, iterations=1)	Dilatation
cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)	Ouverture (érosion + dilatation)
cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)	Fermeture (dilatation + érosion)
cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)	Gradient morphologique
cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)	Top hat
cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)	Black hat

```
# Kernels morphologiques
cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))    Rectangle
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5)) Ellipse
cv2.getStructuringElement(cv2.MORPH_CROSS, (5,5))   Croix
```

— DÉTECTION DE CONTOURS —

```
cv2.Canny(img_gray, threshold1=100,
          threshold2=200)           Détection de contours Canny
cv2.Canny(img_gray, 50, 150,
          apertureSize=3)           Avec taille d'ouverture Sobel
cv2.Laplacian(img_gray, cv2.CV_64F) Laplacien
cv2.Sobel(img_gray, cv2.CV_64F, 1, 0,
          ksize=3)                  Sobel axe X
cv2.Sobel(img_gray, cv2.CV_64F, 0, 1,
          ksize=3)                  Sobel axe Y
cv2.Scharr(img_gray, cv2.CV_64F, 1, 0) Scharr (plus précis que Sobel 3x3)
```

EXEMPLES :

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)
cv2.imshow("Contours", edges)
cv2.waitKey(0)
```

[img] Seuillage – threshold / adaptiveThreshold / inRange

DESCRIPTION : Binariser une image ou isoler une plage de couleurs.
 POURQUOI : Segmentation, détection d'objets, masques.

— SEUILLAGE SIMPLE —

```
_, binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_BINARY)
_, binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_BINARY_INV)
_, binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_TRUNC)
_, binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_TOZERO)
_, binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_TOZERO_INV)
_, otsu = cv2.threshold(img_gray, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)    Seuil automatique Otsu
_, tri = cv2.threshold(img_gray, 0, 255,
                       cv2.THRESH_BINARY + cv2.THRESH_TRIANGLE) Méthode Triangle
```

— SEUILLAGE ADAPTATIF —

```
adapt = cv2.adaptiveThreshold(
    img_gray, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,           Moyenne locale
    cv2.THRESH_BINARY, 11, 2)           blockSize, C
adapt = cv2.adaptiveThreshold(
    img_gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,      Gaussienne locale
    cv2.THRESH_BINARY, 11, 2)
```

— MASQUE PAR PLAGE DE COULEUR —

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
bas = np.array([40, 40, 40])
haut = np.array([80, 255, 255])
masque = cv2.inRange(hsv, bas, haut)    Masque binaire (blanc = dans la plage)
resultat = cv2.bitwise_and(img, img,
                             mask=masque) Applique le masque
```

EXEMPLES :

```
# Détection d'une couleur
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
masque_rouge1 = cv2.inRange(hsv, np.array([0,120,70]), np.array([10,255,255]))
masque_rouge2 = cv2.inRange(hsv, np.array([170,120,70]), np.array([180,255,255]))
masque_rouge = cv2.add(masque_rouge1, masque_rouge2)
objet_rouge = cv2.bitwise_and(img, img, mask=masque_rouge)
```

[img] Contours – findContours / drawContours

DESCRIPTION : Détecter et analyser les contours d'objets dans une image binaire.
 POURQUOI : Détecter des formes, mesurer des objets, extraire des ROI.

— TROUVER LES CONTOURS —	
<code>contours, hierarchy = cv2.findContours(</code> <code> binary, cv2.RETR_EXTERNAL,</code> <code> cv2.CHAIN_APPROX_SIMPLE)</code>	Mode : RETR_EXTERNAL / RETR_LIST / RETR_TREE Méthode : CHAIN_APPROX_SIMPLE / NONE
<code>cv2.RETR_EXTERNAL</code> <code>cv2.RETR_LIST</code> <code>cv2.RETR_CCOMP</code> <code>cv2.RETR_TREE</code>	Contours externes seulement Tous les contours, pas de hiérarchie 2 niveaux de hiérarchie Hiérarchie complète
<code>cv2.CHAIN_APPROX_SIMPLE</code> <code>cv2.CHAIN_APPROX_NONE</code>	Comprime les segments (moins de points) Tous les points du contour

— DESSINER LES CONTOURS —	
<code>cv2.drawContours(img, contours, -1,</code> <code> (0,255,0), 2)</code>	Dessine tous les contours
<code>cv2.drawContours(img, contours, 0,</code> <code> (0,255,0), 2)</code>	Dessine le premier contour
<code>cv2.drawContours(img, [cnt], -1,</code> <code> (255,0,0), -1)</code>	Dessine un contour spécifique -1 = rempli

— PROPRIÉTÉS DES CONTOURS —	
<code>cv2.contourArea(cnt)</code>	Aire du contour (pixels ²)
<code>cv2.arcLength(cnt, closed=True)</code>	Périmètre
<code>cv2.boundingRect(cnt)</code>	Bounding box (x, y, w, h)
<code>cv2.minEnclosingCircle(cnt)</code>	Cercle minimal → (centre, rayon)
<code>cv2.minAreaRect(cnt)</code>	Rectangle minimal orienté
<code>cv2.convexHull(cnt)</code>	Enveloppe convexe
<code>cv2.isContourConvex(cnt)</code>	True si convexe
<code>cv2.moments(cnt)</code>	Moments du contour
<code>M = cv2.moments(cnt)</code>	
<code>cx = int(M["m10"] / M["m00"])</code>	Centroïde X
<code>cy = int(M["m01"] / M["m00"])</code>	Centroïde Y
<code>cv2.approxPolyDP(cnt, epsilon, True)</code>	Approximation polygonale
<code>epsilon = 0.02 * cv2.arcLength(cnt, True)</code>	Epsilon typique

— FILTRER LES CONTOURS —

```
# Garder les contours de plus de 500 pixels²
grands = [c for c in contours if cv2.contourArea(c) > 500]

# Trier par aire décroissante
tries = sorted(contours, key=cv2.contourArea, reverse=True)
```

EXEMPLES :

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(binary,
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for cnt in contours:
    if cv2.contourArea(cnt) > 200:
        x, y, w, h = cv2.boundingRect(cnt)
        cv2.rectangle(img, (x,y), (x+w,y+h), (0,255,0), 2)
```

[draw] Dessin sur image – rectangle / circle / line / putText

DESCRIPTION : Dessiner des formes géométriques et du texte sur une image.
POURQUOI : Annotations, visualisation de détections, debug.

— FORMES —	
<code>cv2.rectangle(img, (x1,y1), (x2,y2),</code> <code> color, thickness)</code>	Rectangle
<code>cv2.rectangle(img, (x,y), (x+w,y+h),</code> <code> (0,255,0), 2)</code>	Bounding box verte épaisseur 2
<code>cv2.rectangle(img, (x,y), (x+w,y+h),</code> <code> (0,255,0), -1)</code>	Rectangle rempli (thickness=-1)
<code>cv2.circle(img, (cx,cy), radius,</code> <code> color, thickness)</code>	Cercle
<code>cv2.circle(img, (cx,cy), 5,</code> <code> (0,0,255), -1)</code>	Point rempli
<code>cv2.ellipse(img, (cx,cy),</code> <code> (a,b), angle, 0, 360, color, thickness)</code>	Ellipse

cv2.line(img, (x1,y1), (x2,y2), color, thickness)	Ligne
cv2.arrowedLine(img, (x1,y1), (x2,y2), color, thickness)	Flèche
cv2.polylines(img, [pts], isClosed, color, thickness)	Polygone ouvert ou fermé
cv2.fillPoly(img, [pts], color)	Polygone rempli

— TEXTE —

cv2.putText(img, "texte", (x, y), cv2.FONT_HERSHEY_SIMPLEX, fontScale=1.0, color=(255,255,255), thickness=2, lineType=cv2.LINE_AA)	Position du coin bas-gauche Police Taille relative Couleur BGR Épaisseur Anti-aliasing
# Polices disponibles :	
cv2.FONT_HERSHEY_SIMPLEX	Simple
cv2.FONT_HERSHEY_COMPLEX	Complexe
cv2.FONT_HERSHEY_DUPLEX	Double trait
cv2.FONT_HERSHEY_TRIPLEX	Triple trait
cv2.FONT_HERSHEY_SCRIPT_SIMPLEX	Cursive
cv2.FONT_ITALIC	Italique (combinable avec)
# Mesurer le texte avant de l'écrire (w_txt, h_txt), baseline = cv2.getTextSize("Mon texte", cv2.FONT_HERSHEY_SIMPLEX, 1.0, 2)	

COULEURS BGR COURANTES :

(0, 0, 0)	Noir	(255, 255, 255)	Blanc
(0, 0, 255)	Rouge	(0, 255, 0)	Vert
(255, 0, 0)	Bleu	(0, 255, 255)	Jaune
(255, 0, 255)	Magenta	(0, 165, 255)	Orange

EXEMPLES :

```
# Dessiner un label au-dessus d'une bounding box
label = f"Personne (92%)"
(tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 1)
cv2.rectangle(img, (x, y-th-8), (x+tw, y), (0,255,0), -1)
cv2.rectangle(img, (x, y), (x+w, y+h), (0,255,0), 2)
cv2.putText(img, label, (x, y-4),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,0), 1, cv2.LINE_AA)
```

[vid] Vidéo – VideoCapture / VideoWriter

DESCRIPTION : Lire depuis une caméra ou un fichier, écrire une vidéo.
POURQUOI : Traitement vidéo en temps réel, enregistrement, analyse.

— LECTURE VIDÉO —

cap = cv2.VideoCapture("video.mp4")	Depuis un fichier
cap = cv2.VideoCapture(0)	Webcam 0 (première caméra)
cap = cv2.VideoCapture(1)	Deuxième caméra
cap = cv2.VideoCapture("rtsp://...")	Flux RTSP
cap = cv2.VideoCapture("http://...stream")	Flux HTTP
cap.isOpened()	True si ouvert
ret, frame = cap.read()	Lit une frame → (succès, frame)
cap.release()	Libère la caméra/fichier

— PROPRIÉTÉS VideoCapture —

cap.get(cv2.CAP_PROP_FRAME_WIDTH)	Largeur
cap.get(cv2.CAP_PROP_FRAME_HEIGHT)	Hauteur
cap.get(cv2.CAP_PROP_FPS)	FPS
cap.get(cv2.CAP_PROP_FRAME_COUNT)	Nombre total de frames
cap.get(cv2.CAP_PROP_POS_FRAMES)	Frame courante (position)
cap.get(cv2.CAP_PROP_POS_MSEC)	Position en millisecondes
cap.get(cv2.CAP_PROP_FOURCC)	Codec
cap.set(cv2.CAP_PROP_POS_FRAMES, 100)	Sauter à la frame 100
cap.set(cv2.CAP_PROP_POS_MSEC, 5000)	Sauter à 5 secondes
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)	Définir largeur (caméra)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)	Définir hauteur (caméra)
cap.set(cv2.CAP_PROP_FPS, 30)	Définir FPS (caméra)

— ÉCRITURE VIDÉO —

```
fourcc = cv2.VideoWriter_fourcc(*"mp4v")    Codec MP4
fourcc = cv2.VideoWriter_fourcc(*"XVID")     Codec AVI
fourcc = cv2.VideoWriter_fourcc(*"MJPG")     Motion JPEG
fourcc = cv2.VideoWriter_fourcc(*"avc1")     H.264 (Mac)
fourcc = cv2.VideoWriter_fourcc(*"H264")     H.264 (Linux)
fourcc = cv2.VideoWriter_fourcc(*"VP90")     VP9 WebM

out = cv2.VideoWriter(
    "sortie.mp4", fourcc, fps=30.0,
    frameSize=(largeur, hauteur))            Créer un writer
out = cv2.VideoWriter(
    "sortie.mp4", fourcc, 30.0,
    (w, h), isColor=True)                   Avec couleur explicite
out.write(frame)                             Écrire une frame
out.release()                               Finaliser le fichier
```

EXEMPLES :

```
cap = cv2.VideoCapture("entree.mp4")
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("sortie.mp4", fourcc, fps, (w, h))

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray_bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
    out.write(gray_bgr)

cap.release()
out.release()

# Webcam temps réel
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Webcam", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
cv2.destroyAllWindows()
```

[feat] Détection de visages et objets – CascadeClassifier / DNN	
---	--

DESCRIPTION : Détecter des visages et objets avec les classifieurs intégrés.
POURQUOI : Détection rapide sans modèle externe.

— CASCADE CLASSIFIER —

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_eye.xml")
body_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_fullbody.xml")
profile_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_profileface.xml")

faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30, 30),
    maxSize=(300, 300))

for (x, y, w, h) in faces:
```

Facteur de réduction de l'image
Nombre min de détections voisines
Taille minimale du visage
Taille maximale

```
cv2.rectangle(img, (x,y), (x+w,y+h), (255,0,0), 2)
```

— RÉSEAU DE NEURONES (DNN MODULE) —

```
# Charger un modèle (ONNX, TensorFlow, Caffe, Darknet)
net = cv2.dnn.readNet("model.onnx")
net = cv2.dnn.readNetFromCaffe("deploy.prototxt", "model.caffemodel")
net = cv2.dnn.readNetFromTensorflow("model.pb")
net = cv2.dnn.readNetFromDarknet("yolo.cfg", "yolo.weights")
net = cv2.dnn.readNetFromONNX("model.onnx")

# Préparer l'entrée
blob = cv2.dnn.blobFromImage(
    img,
    scalefactor=1/255.0,
    size=(416, 416),
    mean=(0, 0, 0),
    swapRB=True,
    crop=False)

# Normalisation
# Taille d'entrée du modèle
# Soustraction de moyenne
# Swap BGR→RGB

net.setInput(blob)
outputs = net.forward()
outputs = net.forward(output_layers)

# Inférence (1 sortie)
# Inférence multi-sorties

# Accélération GPU
net.setPreferableBackend(cv2.dnn.DNN_BACKEND_CUDA)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CUDA)
net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
```

[feat] Features et correspondances – ORB / SIFT / BFMatcher

DESCRIPTION : Détecter et faire correspondre des points caractéristiques.
POURQUOI : Reconnaissance d'objets, assemblage de panoramas, tracking.

— DÉTECTEURS / DESCRIPTEURS —

```
orb = cv2.ORB_create(nfeatures=500)
sift = cv2.SIFT_create()
akaze = cv2.AKAZE_create()
brisk = cv2.BRISK_create()

kp, des = orb.detectAndCompute(gray, None)
kp = orb.detect(gray, None)
des = orb.compute(gray, kp)
```

ORB (rapide, libre de droits)
SIFT (précis, lent)
AKAZE
BRISK

Points clés + descripteurs
Points clés seulement
Descripteurs seulement

— APPARIEMENT —

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING,
    crossCheck=True)
bf = cv2.BFMatcher(cv2.NORM_L2)
matches = bf.match(des1, des2)
matches = bf.knnMatch(des1, des2, k=2)

# Brute force (ORB/BRISK)
# Pour SIFT/SURF
# Appariement simple
# K meilleurs appariements

# Filtre de Lowe (ratio test)
bons = [m for m, n in matches
    if m.distance < 0.75 * n.distance]

# FLANN (plus rapide pour grands ensembles)
index_params = dict(algorithm=1, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1, des2, k=2)

# FLANN_INDEX_KDTREE
```

— VISUALISATION —

```
img_kp = cv2.drawKeypoints(gray, kp, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
img_matches = cv2.drawMatches(img1, kp1, img2, kp2,
    bons[:20], None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

— HOMOGRAPHIE —

```
pts_src = np.float32([kp1[m.queryIdx].pt for m in bons]).reshape(-1,1,2)
pts_dst = np.float32([kp2[m.trainIdx].pt for m in bons]).reshape(-1,1,2)
H, mask = cv2.findHomography(pts_src, pts_dst,
    cv2.RANSAC, ransacReprojThreshold=5.0)
```



```
result = cv2.warpPerspective(img1, H, (w, h))
```

[feat] Traitement d'images avancé – histogrammes / transformées

DESCRIPTION : Analyse et amélioration d'images par histogramme et transformées.

— HISTOGRAMMES —

```
hist = cv2.calcHist([img_gray], [0], None,
                    [256], [0,256])           Histogramme niveaux de gris
hist = cv2.calcHist([img], [0,1,2], None,
                    [8,8,8], [0,256]*3)       Histogramme 3D couleur

cv2.equalizeHist(img_gray)                   Égalisation d'histogramme
clahe = cv2.createCLAHE(clipLimit=2.0,
                        tileGridSize=(8,8))   CLAHE (adaptative)
clahe.apply(img_gray)                       Applique CLAHE
```

— OPÉRATIONS ARITHMÉTIQUES —

```
cv2.add(img1, img2)                         Addition saturée
cv2.subtract(img1, img2)                     Soustraction saturée
cv2.multiply(img1, img2)                     Multiplication
cv2.divide(img1, img2)                       Division
cv2.addWeighted(img1, 0.7, img2, 0.3, 0)    Fusion pondérée (alpha blending)
cv2.absdiff(img1, img2)                     Différence absolue
cv2.bitwise_and(img1, img2)                 ET binaire
cv2.bitwise_or(img1, img2)                  OU binaire
cv2.bitwise_xor(img1, img2)                 XOR binaire
cv2.bitwise_not(img)                        NOT binaire
```

— TRANSFORMÉE DE FOURIER —

```
dft = cv2.dft(np.float32(img_gray),
              flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)
magnitude = cv2.magnitude(dft_shift[:, :, 0],
                          dft_shift[:, :, 1])
spectrum = 20 * np.log(magnitude + 1)
```

— REMPLISSAGE ET INPAINTING —

```
cv2.floodFill(img, mask=None,
              seedPoint=(x,y), newVal=(255,0,0))  Remplissage par diffusion
cv2.inpaint(img, masque, inpaintRadius=3,
            flags=cv2.INPAINT_TELEA)              Restauration (suppression d'objets)
cv2.inpaint(img, masque, 3,
            cv2.INPAINT_NS)                       Navier-Stokes
```

— DISTANCE ET SQUELETTISATION —

```
dist = cv2.distanceTransform(binary,
                             cv2.DIST_L2, 3)     Transformée de distance
cv2.normalize(dist, dist, 0, 1.0,
              cv2.NORM_MINMAX)                    Normaliser pour affichage
```

— WATERSHED (SEGMENTATION) —

```
cv2.watershed(img, markers)                    Segmentation par watershed
```

[feat] Tracking – suivi d'objets entre frames

DESCRIPTION : Suivre des objets en mouvement dans une séquence vidéo.

POURQUOI : Surveillance, analyse de mouvement, sports analytics.

— FLUX OPTIQUE —

```
# Lucas-Kanade (points spécifiques)
pts_next, status, err = cv2.calcOpticalFlowPyrLK(
    prev_gray, curr_gray,
    pts_prev, None,
    winSize=(15,15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS |
              cv2.TERM_CRITERIA_COUNT, 10, 0.03))

# Farneback (flux dense)
flow = cv2.calcOpticalFlowFarneback(
```

```
prev_gray, curr_gray, None,
pyr_scale=0.5, levels=3,
winsize=15, iterations=3,
poly_n=5, poly_sigma=1.2,
flags=0)
```

— TRACKERS INTÉGRÉS —

tracker = cv2.TrackerCSRT_create()	CSRT (précis, lent)
tracker = cv2.TrackerKCF_create()	KCF (rapide)
tracker = cv2.TrackerMIL_create()	MIL
tracker = cv2.legacy.TrackerMOSSE_create()	MOSSE (très rapide)
bbox = (x, y, w, h)	Bounding box initiale
tracker.init(first_frame, bbox)	Initialise le tracker
ok, bbox = tracker.update(frame)	Met à jour → (succès, bbox)

— SOUSTRACTION DE FOND —

```
bg = cv2.createBackgroundSubtractorMOG2(
    history=500, varThreshold=16,
    detectShadows=True)
bg = cv2.createBackgroundSubtractorKNN()

fg_mask = bg.apply(frame)      Masque de premier plan
bg_img = bg.getBackgroundImage() Image de fond estimée
```

[img] Interopérabilité – NumPy / PIL / Matplotlib

DESCRIPTION : Convertir entre OpenCV, NumPy, Pillow et Matplotlib.
POURQUOI : Interopérabilité dans les pipelines de traitement.

— OPENCV ↔ NUMPY —

# Une image OpenCV EST un array NumPy	
img_np = np.array(img)	Copie vers NumPy
img_cv = img_np.copy()	Copie depuis NumPy
pixel = img[y, x]	Accès à un pixel (B, G, R)
pixel = img[y, x, 0]	Canal bleu d'un pixel
img[y, x] = [255, 0, 0]	Modifier un pixel

— OPENCV ↔ PIL/PILLOW —

```
from PIL import Image
# PIL → OpenCV
img_cv = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)
# OpenCV → PIL
pil_img = Image.fromarray(cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB))
```

— OPENCV ↔ MATPLOTLIB —

```
import matplotlib.pyplot as plt
# Afficher dans Matplotlib (BGR → RGB)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()

# Image en niveaux de gris
plt.imshow(gray, cmap="gray")
plt.show()
```

— COPIE D'IMAGE —

img_copie = img.copy()	Copie profonde (important !)
img_ref = img	Référence (modifications partagées !)

===== RÉCAPITULATIF OPENCV (CV2) =====

TÂCHE	FONCTION PRINCIPALE
Lire une image	cv2.imread("f.jpg")
Sauvegarder	cv2.imwrite("f.jpg", img)
Afficher	cv2.imshow("win", img) + cv2.waitKey(0)
BGR → Gris	cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
BGR → HSV	cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
BGR → RGB	cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

Redimensionner	cv2.resize(img, (w, h))
Recadrer	img[y1:y2, x1:x2]
Flip	cv2.flip(img, 1)
Rotation 90°	cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
Rotation libre	cv2.getRotationMatrix2D + cv2.warpAffine
Flou gaussien	cv2.GaussianBlur(img, (5,5), 0)
Flou médian	cv2.medianBlur(img, 5)
Flou bilatéral	cv2.bilateralFilter(img, 9, 75, 75)
Détection de contours	cv2.Canny(gray, 100, 200)
Érosion / Dilatation	cv2.erode / cv2.dilate(img, kernel)
Seuillage Otsu	cv2.threshold(gray, 0, 255, THRESH_OTSU)
Seuillage adaptatif	cv2.adaptiveThreshold(...)
Masque couleur HSV	cv2.inRange(hsv, bas, haut)
Trouver les contours	cv2.findContours(binary, RETR_EXTERNAL, ...)
Dessiner rectangle	cv2.rectangle(img, pt1, pt2, color, thickness)
Dessiner cercle	cv2.circle(img, center, radius, color, -1)
Écrire du texte	cv2.putText(img, text, org, font, scale, color)
Lire une vidéo	cv2.VideoCapture("video.mp4")
Lire la webcam	cv2.VideoCapture(0)
Écrire une vidéo	cv2.VideoWriter("out.mp4", fourcc, fps, (w,h))
Détecter des visages	CascadeClassifier + detectMultiScale
Réseau de neurones	cv2.dnn.readNet + blobFromImage + forward
Features ORB	cv2.ORB_create + detectAndCompute
Appariement features	cv2.BFMatcher + match/knnMatch
Homographie	cv2.findHomography + warpPerspective
Flux optique	cv2.calcOpticalFlowPyrLK / Farneback
Soustraction de fond	cv2.createBackgroundSubtractorMOG2
Histogramme	cv2.calcHist / equalizeHist / CLAHE
Alpha blending	cv2.addWeighted(img1, a, img2, b, 0)
Opérations binaires	cv2.bitwise_and/or/xor/not + mask=
Inpainting	cv2.inpaint(img, mask, 3, INPAINT_TELEA)

— ATTENTION : CONVENTIONS OPENCV —

BGR (pas RGB)	cv2.imread charge en Blue-Green-Red
(largeur, hauteur)	cv2.resize, VideoCapture frameSize → (w, h)
[y, x]	NumPy indexe ligne (y) puis colonne (x)
uint8 0-255	Type par défaut des images
H : 0-179 en HSV	OpenCV utilise 0-179 pour la teinte (pas 0-360)
thickness=-1	Remplissage pour rectangle, circle, etc.

=====